

Active Learning System for Bacterial Colony Classification

1. Technical Approach

The goal is a production-ready service that is accurate, scalable, and dramatically reduces manual labeling. Here's how we can build it.

- **The Backbone:** DINOv3 is an excellent choice. It's a state-of-the-art self-supervised vision transformer that excels as a feature extractor for detection and classification.
- **A Better Detection Head:** While DINOv3 provides the powerful "eyes" of the system, the "hands" that draw bounding boxes and classify colonies must be equally capable. I recommend building on the latest DETR (Detection Transformer) variants, specifically DEIMv2 or RT-DETRv2. These architectures are a significant step beyond traditional methods:
 - They are significantly faster and more accurate.
 - They integrate more seamlessly with DINOv3's transformer architecture.
- **The Active Learning Loop:** The core of our system will be a sophisticated HITL engine that proactively selects the most valuable images for you to review. It will use:
 - Ensemble Disagreement: Deploying a small ensemble of models to measure "epistemic" uncertainty—the model's uncertainty about what it doesn't know.
 - Hybrid Query Strategies: Combining uncertainty metrics with diversity sampling (like CoreSet or BADGE) to ensure the model doesn't just ask for the same type of ambiguous colony repeatedly.
- **The Inference Service:** All of this will be wrapped in a secure, high-performance API server built with FastAPI and containerized with Docker. This ensures easy deployment in your environment and seamless integration into your existing workflows.

2. Quote for Turnkey System Development

Phase	Description	Duration	Cost (USD)
1	Architecture Design & Specification: Collaborate to design the exact API, data schemas, HITL workflow, and model serving architecture.	1 week	1000
2	Core System Implementation: Build the DINOv3 + DEIMv2 detection model, implement all uncertainty/ensemble/HITL logic, and create the core Python package.	2~3 weeks	2500
3	Inference Server & Integration: Develop the FastAPI server, create Docker containers, implement model versioning and retraining pipeline.	1 week	1000
4	Testing, Deployment & Documentation: Rigorous testing on your infrastructure (VPN/on-prem), full system documentation, and a 2-hour training session for your team.	1 week	1000
5	12-Month Maintenance & Support (Optional): Includes bug fixes, minor updates, and direct support for your engineering team.	12 months	1000

3. Architecture Comparison

Architecture	Key Strengths	Weaknesses	Suitability
DINOv3 + DEIMv2	State-of-the-art accuracy, excellent real-time speed, perfect synergy between backbone and head, excellent for dense prediction tasks.	More complex architecture than YOLO.	Optimal. Maximizes accuracy and efficiency, leading to lower long-term costs.
DINOv3 + RT-DETR	A highly optimized transformer-based detector proven to be real-time, a safe and performant baseline.	Slightly less performant than DEIMv2 on some benchmarks.	Excellent. A fantastic and reliable choice.
Current Adaptive-DETR	Good performance, custom-built for your use case.	Lacks the latest speed and accuracy innovations, training can be slower.	Good, but not best. A capable system that would benefit from an upgrade.
YOLO (v10/v11)	Very fast inference, mature ecosystem, easy to deploy.	While powerful, general-purpose models often lag	Good for speed, less for ultimate accuracy.

		behind transformer-based specialists in accuracy for complex tasks.	
--	--	---	--

By adopting DINOv3 with a DEIMv2 or RT-DETR head, we can improve your current system, ensure it remains a competitive asset.

4. Delivery

1) Phase 1: Architecture Design & Specification (1 week)

Objective: Collaborate to define the exact system boundaries, API contracts, data schemas, and workflow so that development proceeds without ambiguity.

- System Architecture Diagram – showing all components: inference pipeline, uncertainty module, HITL queue, annotation tool, retraining loop, model versioning, and deployment environment.
- API Specification – OpenAPI / Swagger definition of the inference service (endpoints, request/response formats, error codes).
- Data Schema Document – JSON schemas for:
 - Input images (formats, size limits).
 - Inference output (boxes, class probabilities, uncertainty scores).
 - Annotation import/export (Label Studio project config).
 - Retraining trigger metadata.
- Integration Guide – pseudo-code snippets showing how your existing DINOv3 model will be wrapped to fit into the new pipeline (without exposing your source code).
- Project Roadmap – detailed 6 weeks plan with milestones and acceptance criteria.

2) Phase 2: Core Active Learning Module (2~3 weeks)

Objective: Build the Python package that contains all active learning logic, uncertainty estimators, query strategies, model calibration, and pseudo-labeling. This module will be designed to be plugged into your inference pipeline without exposing your code.

- Python package (active_learning_tool/) with:
 - uncertainty.py** – functions to compute entropy, margin, MC-dropout, ensemble disagreement.
 - query_strategy.py** – implementation of hybrid strategies (CoreSet, BADGE, etc.).

`calibration.py` – temperature scaling, Expected Calibration Error (ECE) calculation.

`pseudo_label.py` – confidence-based pseudo-labeling with filters.

`orchestrator.py` – main loop that loads a batch of unlabeled images, runs inference (calls your model), scores uncertainty, selects top K, exports to annotation queue, and merges new labels.

- Example integration script (`example_integration.py`) – shows how to call your existing DINOv3 model from within the orchestrator.

You will replace the mock model with your real one.

- Unit tests for each component.
- Documentation (inline + Markdown) explaining all parameters and expected data formats.

3) Phase 3: Inference Server & HITL Integration (1 week)

Objective: Package the active learning module into a production-ready inference service, integrate with an annotation tool (Label Studio), and implement the retraining pipeline.

- FastAPI-based inference server:
 - Endpoint `/predict` – accepts image, returns predictions + uncertainty scores.
 - Endpoint `/uncertain_batch` – accepts multiple images, returns a ranked list of uncertain samples.
 - Endpoint `/feedback` – receives corrected annotations and stores them for retraining.
 - Automatic model loading and versioning (multiple model versions can coexist).
- Docker container – includes the server, active learning module, and dependencies.
 - Environment variables for configuration (model paths, annotation tool URL, thresholds).
- Label Studio integration:
 - Pre-configured Label Studio project (exported as JSON) with your classes (BC, ETB, YM, XSA).
 - Script to automatically push uncertain images from the server into Label Studio as “pre-annotated” tasks (with model predictions pre-filled).

- Script to pull newly labeled tasks, convert them to training format, and store them.
- Retraining pipeline script (retrain_pipeline.sh or Python):
 - Steps: collect new annotations → merge with existing labeled set → retrain classification heads (your model stays frozen) → evaluate on validation set → if performance improves, promote new model version.
 - Can be run manually or as a scheduled job (e.g., daily).
- Basic monitoring dashboard (Streamlit or simple HTML):
 - Shows number of images labeled, uncertainty distribution, accuracy over retraining rounds.

4) Phase 4: Testing, Deployment & Documentation (1 week)

Objective: Ensure the entire system works in your production environment, and your team is trained to operate and maintain it.

- Test report – documenting system performance on your dataset:
 - Accuracy vs. number of labeled images (active learning curve).
 - Calibration plots (ECE before/after temperature scaling).
 - Inference latency (p95, p99) under expected load.
 - Annotation savings estimate (e.g., “70% fewer labels needed to reach 95% accuracy”).
- Full system documentation – includes:
 - Installation and configuration guide.
 - API reference.
 - Label Studio user guide for annotators.
 - Retraining pipeline operation manual.
 - Troubleshooting section.
- 2-hour remote training session (recorded) for your engineering and annotation teams:

- How to deploy the system.
- How to monitor active learning rounds.
- How to add new classes or update models.
- Source code of all deliverables (Python package, server, scripts) – licensed for your internal use only.